

Проектирование информационных систем

Диаграмма состояний
(State diagram)

Классификация диаграмм UML

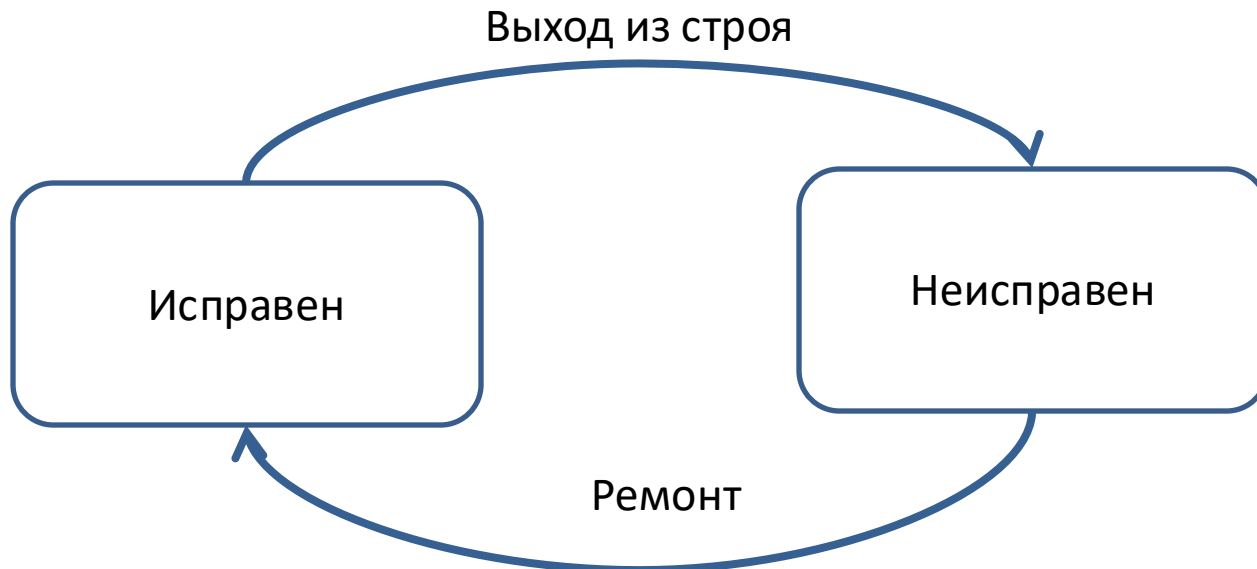


Понятие «автомат» (state machine)

- **«Автомат»** описывает поведение объекта в форме последовательности состояний, которые охватывают все этапы его жизненного цикла, начиная от создания объекта и заканчивая его уничтожением.
- Для описания «автомата» используются понятия: **состояние** и **переход**.
- Отличие между состоянием и переходом: **длительность** нахождения системы в некотором состоянии значительно превышает время, которое затрачивается на переход из одного состояния в другое. Предполагается, что переход осуществляется мгновенно.

Понятие «автомат» (state machine)

- Представим состояния любого технического устройства (телевизор, телефон и др.) на самом общем уровне. Тогда вводятся два состояния «исправен», «неисправен», и два перехода – «выход из строя» и «ремонт», что отображает диаграмма:



Обязательные условия для описания автомата

- Автомат не запоминает историю перемещения из состояния в состояние.
- В каждый момент времени автомат может находиться в одном и только в одном состоянии.
- Время не включается в понятие автомата, т.е. длительность нахождения в состоянии или достижения некоторого состояния не указывается.
- Количество состояний автомата должно быть конечным.
- Автомат не содержит изолированных состояний, т.е. для каждого состояния (кроме начального) должно быть определено предыдущее состояние, каждый переход должен соединять два состояния. Допускается переход из состояния в себя («петля»).
- Автомат не должен содержать конфликтных переходов (т.е. переход в два и более состояний, без указания условий перехода).

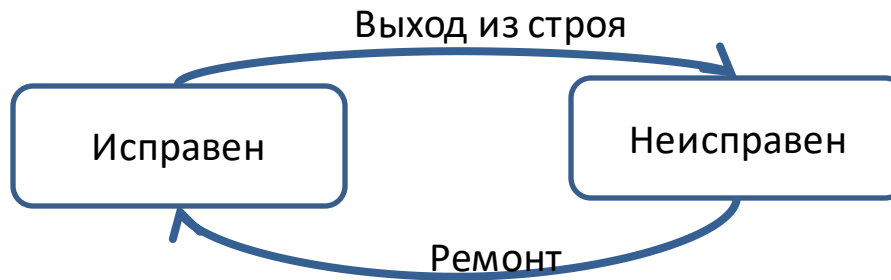


Диаграмма состояний (state diagram). Определение

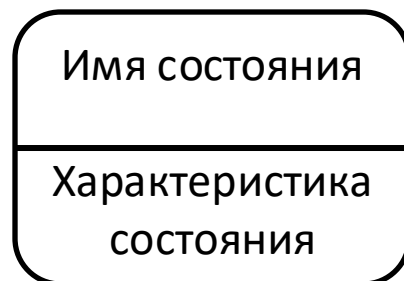
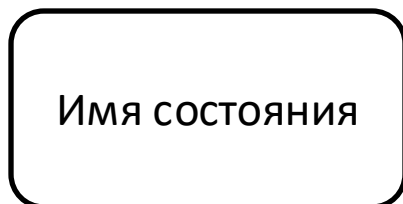
- **Диаграммы состояний** – диаграмма, описывающая возможные последовательности состояний и переходов, которые в совокупности характеризуют поведение элемента модели в течении его жизненного цикла.
- Диаграмма состояний служит для представления динамической составляющей модели системы.
- Основные элементы диаграмм классов: **состояние, переход, событие.**

Диаграмма состояний (state diagram). Определение

- Диаграмма состояний представляет собой ориентированный граф, вершины которого соответствуют состояниям, а дуги - переходам.
- Поведение моделируется как последовательное перемещение по графу состояний от вершины к вершине по связывающим их дугам с учетом их ориентации.

Основные элементы диаграммы состояний. Состояние

- **Состояние** отображается в виде прямоугольника со скругленными углами, внутри которого записывается имя.
- Рекомендуется в качестве имени использовать глаголы в настоящем времени (звонит, печатает, ожидает) или причастия (занято, передано, получено).



Характеристика состояния. Основные действия

<метка-действия '/' выражение-действия>

Стандартные метки:

- **entry** (англ. – вход) - действие, которое выполняется в момент входа в данное состояние (входное действие); Например, создать соединение с базой данных **entry / createConnect()**;
- **exit** (англ. – выход) - действие, которое выполняется в момент выхода из данного состояния (выходное действие); Например, закрыть соединение с базой данных **exit / closeConnect()**;
- **do** (англ. – выполнять) - выполняющаяся деятельность ("do activity") в течение всего времени, пока объект находится в данном состоянии; Находясь в состоянии, объект может бездействовать и ждать наступления некоторого события, а может выполнять длительную операцию. Например, рассчитать допускаемые скорости **do / calculateVdop()**. Допускается указывать несколько операций в виде отдельных строк, каждая из которых начинается с метки **do**, или в виде одной строки, операции в которой отделены друг от друга точкой с запятой;

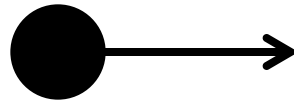
Характеристика состояния. Основные действия

- **defer** (англ. – отложить) - событие, обработка которого предписывается в другом состоянии, но после того, как все операции в текущем будут завершены. Например, отображение на экране сообщения об ошибках в исходных данных **defer / showDataError()**.
- **newTarget** (англ. – новое задание) – внутренний переход, предписывающий обработку новых событий, не покидая текущего состояния. При выполнении внутреннего перехода повторно не выполняются действия при входе или выходе из состояния. Например, временная остановка (прерывание) расчета допускаемых скоростей, **newTarget / pauseCalculateVdop();**

Ввод пароля
entry/установить символы невидимыми exit/установить символы видимыми символ/получить символ помощь/показать помощь

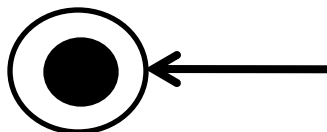
Основные элементы диаграммы состояний. Начальное состояние

- **Начальное состояние** представляет собой частный случай состояния, которое не содержит никаких внутренних действий (псевдосостояния).
- В этом состоянии находится объект по умолчанию в начальный момент времени. Оно служит для указания на диаграмме состояний графической области, от которой начинается процесс изменения состояний.
- Графически начальное состояние в языке UML обозначается в виде закрашенного кружка, из которого может только выходить стрелка, соответствующая переходу.



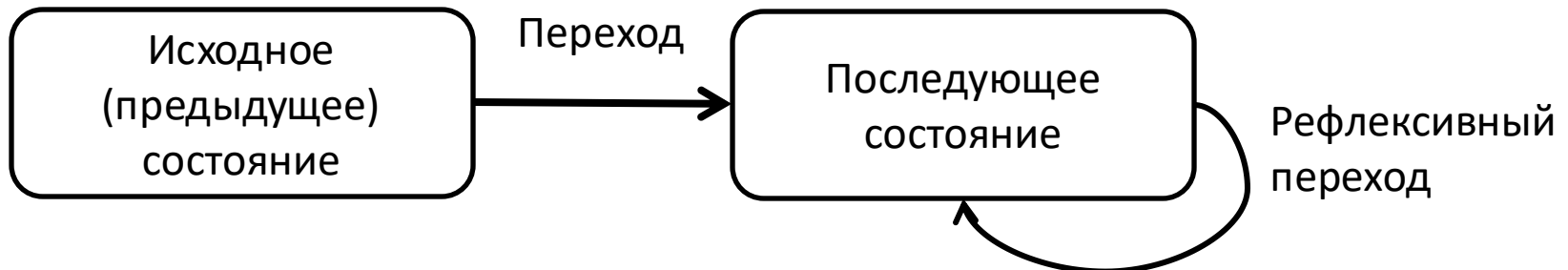
Основные элементы диаграммы состояний. Конечное (финальное) состояние

- **Конечное состояние** представляет собой частный случай состояния, которое также не содержит никаких внутренних действий (псевдосостояния).
- В этом состоянии будет находиться объект по умолчанию после завершения работы автомата в конечный момент времени. Оно служит для указания на диаграмме состояний графической области, в которой завершается процесс изменения состояний или жизненный цикл данного объекта.
- Графически конечное состояние в языке UML обозначается в виде закрашенного кружка, помещенного в окружность, в которую может только входить стрелка, соответствующая переходу.



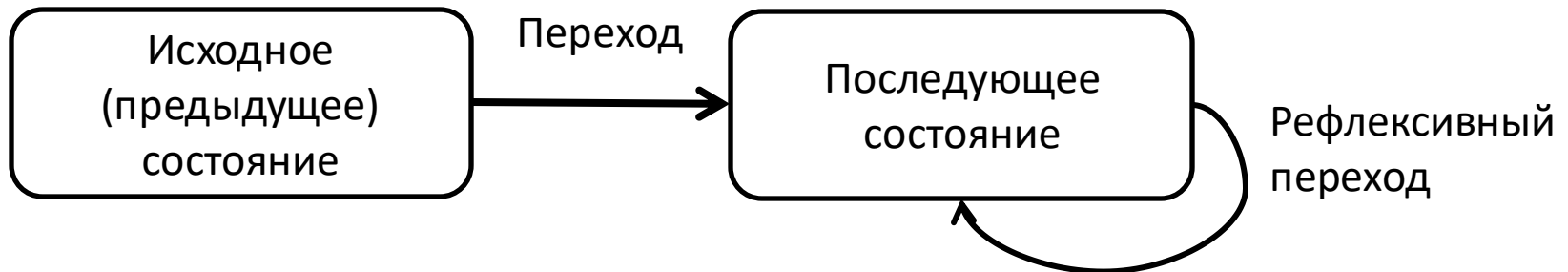
Основные элементы диаграммы состояний. Переход

- **Простой** переход (simple transition) представляет собой отношение между двумя последовательными состояниями, которое указывает на факт смены одного состояния другим.
- Пребывание моделируемого объекта в первом состоянии может сопровождаться выполнением некоторых действий, а переход во второе состояние будет возможен после завершения этих действий, а также после удовлетворения некоторых дополнительных условий.
- В этом случае говорят, что переход **срабатывает**.
- Переход может быть направлен в то же состояние, из которого он выходит. Такой переход называется **рефлексивным**.



Основные элементы диаграммы состояний. Переход

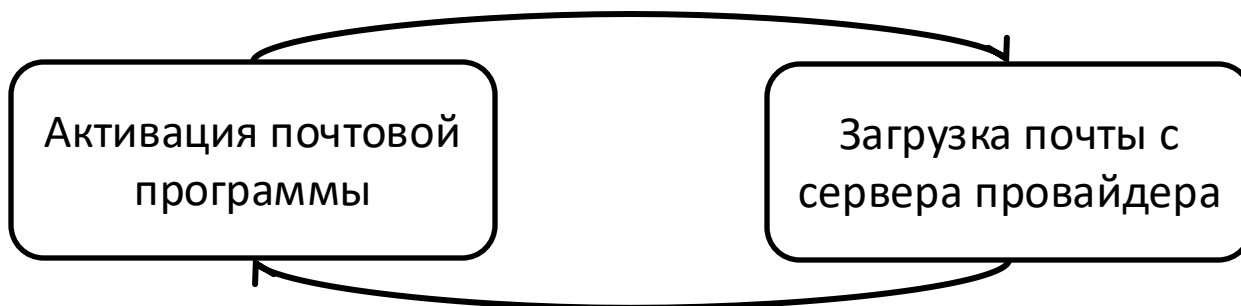
- Переход изображается сплошной линией со стрелкой.
- На переходе указывается **имя события**.
- Переход осуществляется при наступлении некоторого **события**:
 - Окончания выполнения действий (метка do);
 - Получения объектом сообщения;
 - Приема сигнала и т.д.
- или выполнения некоторого **сторожевого условия**, т.е. условие принимает значение «истина».



Основные элементы диаграммы состояний. Переход

- Каждый переход может помечен строкой текста, которая имеет следующий общий формат:
- **<сигнатура события>'['<сторожевое условие>']' /<выражение действия>.**
- При этом сигнатура события описывает некоторое событие с необходимыми аргументами:
- **<сигнатура события> это:**
<имя события>'('<список параметров, разделенных запятыми>')'.

установить телефонное соединение (тел. номер)
[телефонное соединение установлено]



закончить загрузку почты [почтовый ящик на сервере пуст] /
разорвать телефонное соединение (тел. номер)

Основные элементы диаграммы состояний. Событие (event)

- Событие играет роль стимула, который инициирует переход из одного состояния в другое.
- В качестве событий можно рассматривать
 - сигналы,
 - вызовы,
 - окончание фиксированных промежутков времени или
 - моменты окончания выполнения определенных действий.
- Если переход осуществляется в результате выполнения события, то переход называют **триггерным**.
- Если переход не сопровождается текстовой строкой, то переход называется **нетриггерным**, и переход срабатывает после окончания деятельности в рассматриваемом состоянии.

Основные элементы диаграммы состояний. Сторожевое условие

- **Сторожевое условие** (guard condition), если оно есть, всегда записывается в прямых скобках после события-триггера и представляет собой некоторое **булевское выражение**.
- Булевское выражение должно принимать одно из двух взаимно исключающих значений: "**истина**" или "**ложь**". Из контекста диаграммы состояний должна явно следовать семантика этого выражения ($a=b$; $a>b$; $a\neq b$; $a<b$; $a\leq b$; $a\geq b$).
- Если сторожевое условие принимает значение "**истина**", то соответствующий переход может сработать, в результате чего объект перейдет в целевое состояние. Если же сторожевое условие принимает значение "**ложь**", то переход не может сработать, и при отсутствии других переходов объект не может перейти в целевое состояние по этому переходу.
- Однако вычисление истинности сторожевого условия происходит только после **возникновения ассоциированного с ним события-триггера**, инициирующего соответствующий переход.

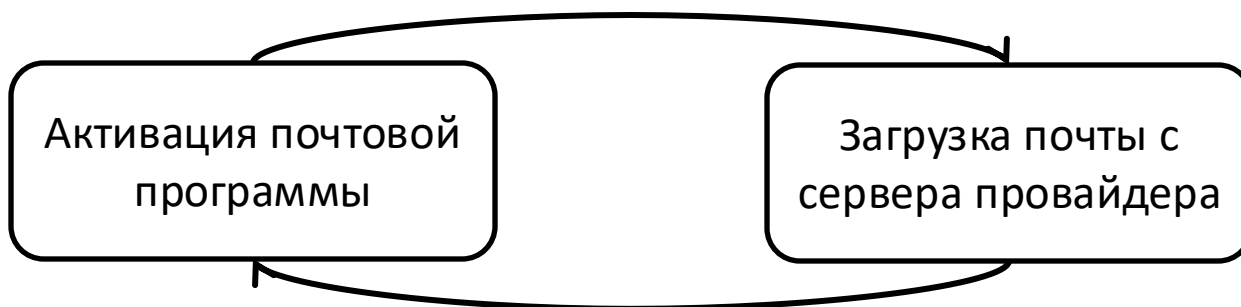
Основные элементы диаграммы состояний. Выражение действия

- **Выражение действия** (action expression) выполняется в том и только в том случае, когда переход срабатывает.
- Представляет собой **атомарную операцию** (достаточно простое вычисление), выполняемую сразу после срабатывания соответствующего перехода до начала каких бы то ни было действий в целевом состоянии.
- Атомарность действия означает, что оно **не может быть прервано никаким другим действием** до тех пор, пока не закончится его выполнение.
- В общем случае, выражение действия может содержать целый список отдельных действий, разделенных символом ";".
- Обязательное требование — все действия из списка должны четко различаться между собой и следовать в порядке их записи.
- На синтаксис записи выражений действия не накладывается никаких ограничений. Главное — их запись должна быть понятна разработчикам модели и программистам. Поэтому чаще всего выражения записывают на одном из языков программирования, который предполагается использовать для реализации модели.

Основные элементы диаграммы состояний. Переход

- Каждый переход может помечен строкой текста, которая имеет следующий общий формат:
- **<сигнатура события>'['<сторожевое условие>']' /<выражение действия>.**
- При этом сигнатура события описывает некоторое событие с необходимыми аргументами:
- **<сигнатура события> это:**
<имя события>'('<список параметров, разделенных запятыми>')'.

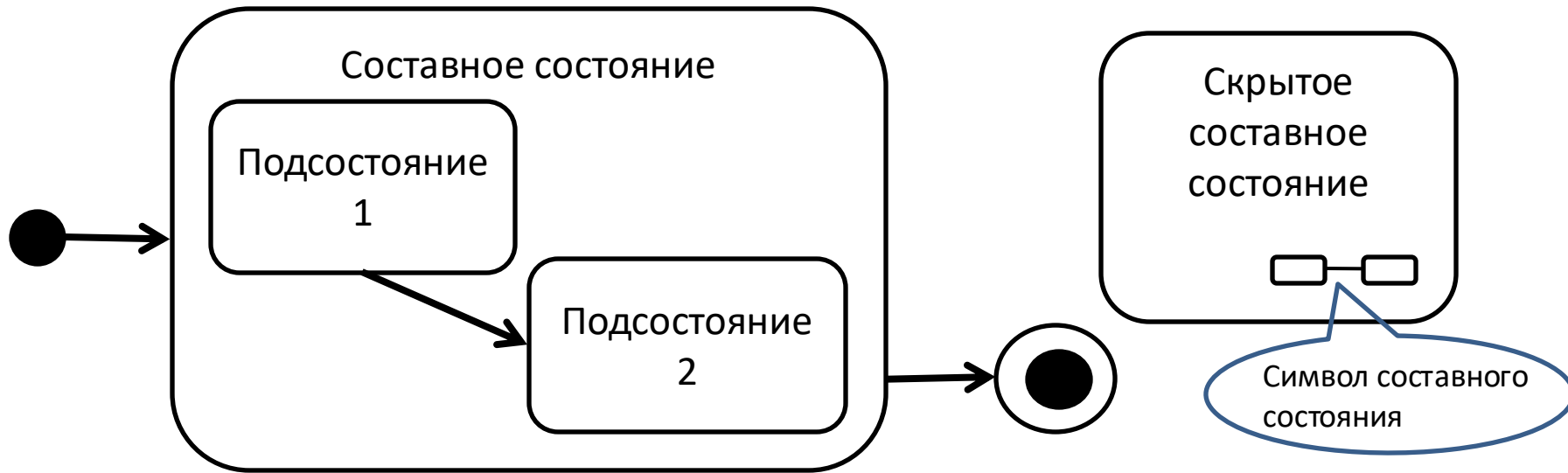
установить телефонное соединение (тел. номер)
[телефонное соединение установлено]



закончить загрузку почты [почтовый ящик на сервере пуст] /
разорвать телефонное соединение (тел. номер)

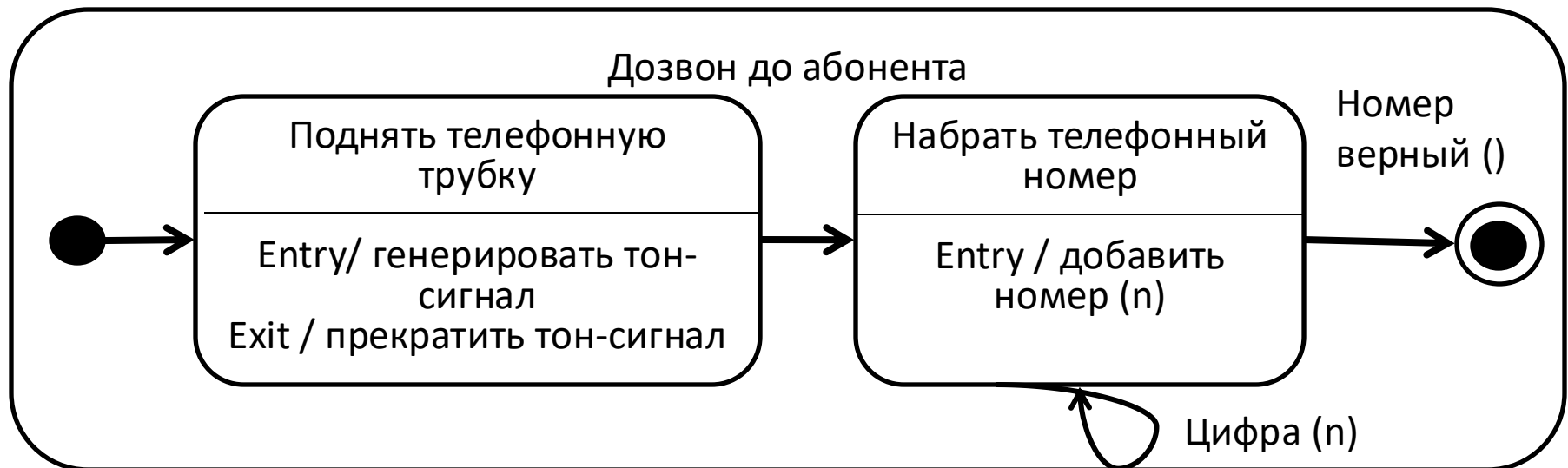
Составное состояние

- **Составное состояние** (composite state) — такое сложное состояние, которое состоит из других вложенных в него состояний.
- Последние будут выступать по отношению к первому как **подсостояния** (substate).
- Графически все вершины диаграммы, которые соответствуют вложенным состояниям, изображаются внутри символа составного состояния.
- В этом случае размеры графического символа составного состояния увеличиваются, так чтобы вместить в себя **все подсостояния**.
- Вложенность подсостояний в UML **не фиксирована**.



Последовательные подсостояния

- **Последовательные подсостояния** (sequential substates) используются для моделирования такого поведения объекта, во время которого в каждый момент времени объект может находиться в одном и только одном подсостоянии.
- Поведение объекта в этом случае представляет собой последовательную смену подсостояний, начиная от начального и заканчивая конечным подсостояниями.



Рекомендации

- По своему назначению диаграмма состояний не является обязательным представлением в модели.
- Диаграмма состояний как бы "присоединяется" к тому элементу, который, по замыслу разработчиков, имеет нетривиальное поведение в течение своего жизненного цикла.
- Наличие у системы нескольких состояний, отличающихся от простой дихотомии "исправен — неисправен", "активен — неактивен", "ожидание — реакция на внешние действия", уже служит признаком необходимости построения диаграммы состояний.
- В качестве начального варианта диаграммы состояний, если нет очевидных соображений по поводу состояний объекта, можно воспользоваться этими суперсостояниями, рассматривая их как составные и уточняя их (детализируя их внутреннюю структуру) по мере рассмотрения логики поведения объекта.

Рекомендации

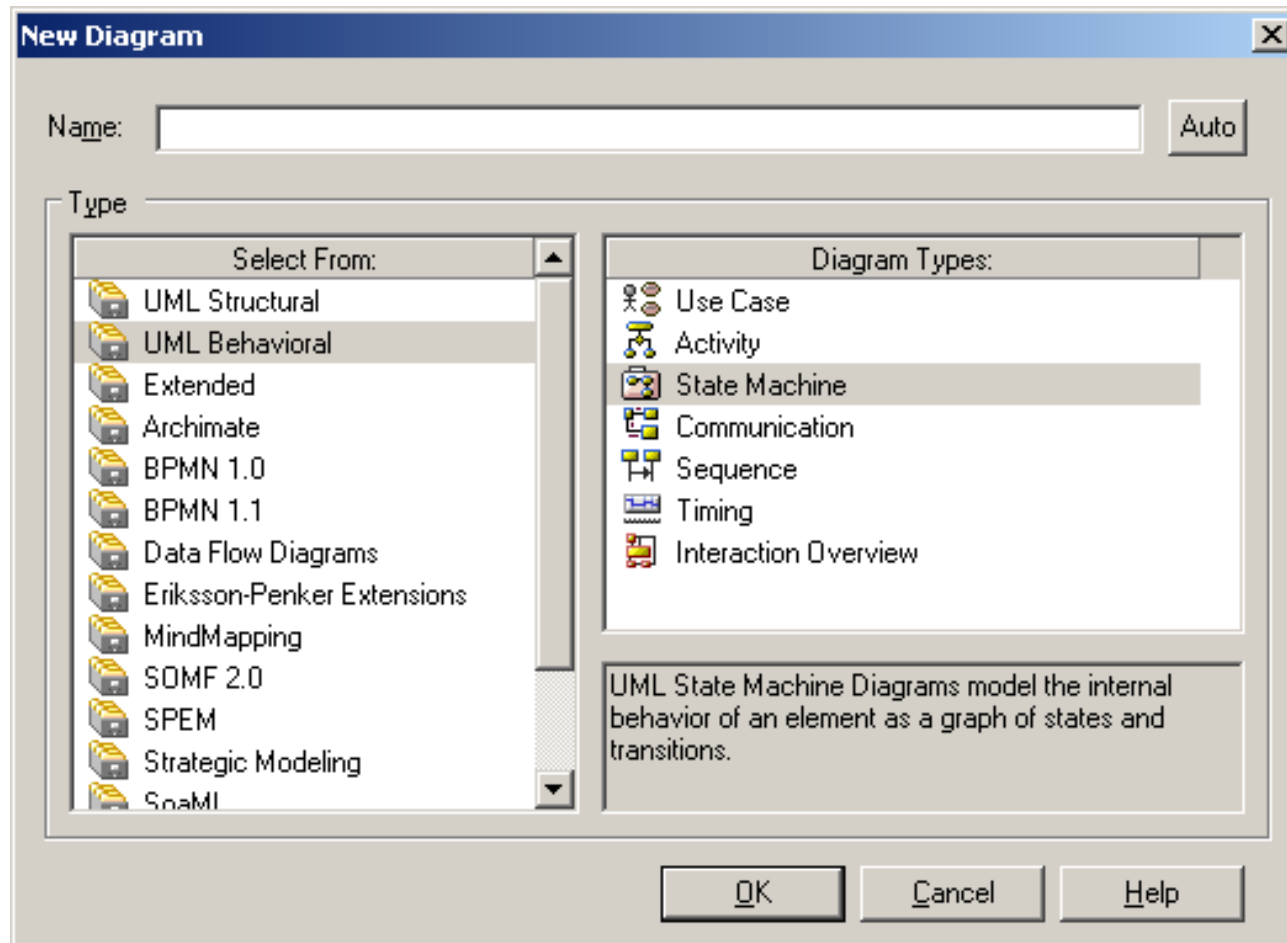
- При разработке диаграммы состояний нужно постоянно следить, чтобы объект в каждый момент мог находиться только в единственном состоянии.
- Если это не так, то данное обстоятельство может быть как следствием ошибки, так и неявным признаком наличия параллельности у поведения моделируемого объекта (данное условие не рассматривается в курсе).

Рекомендации

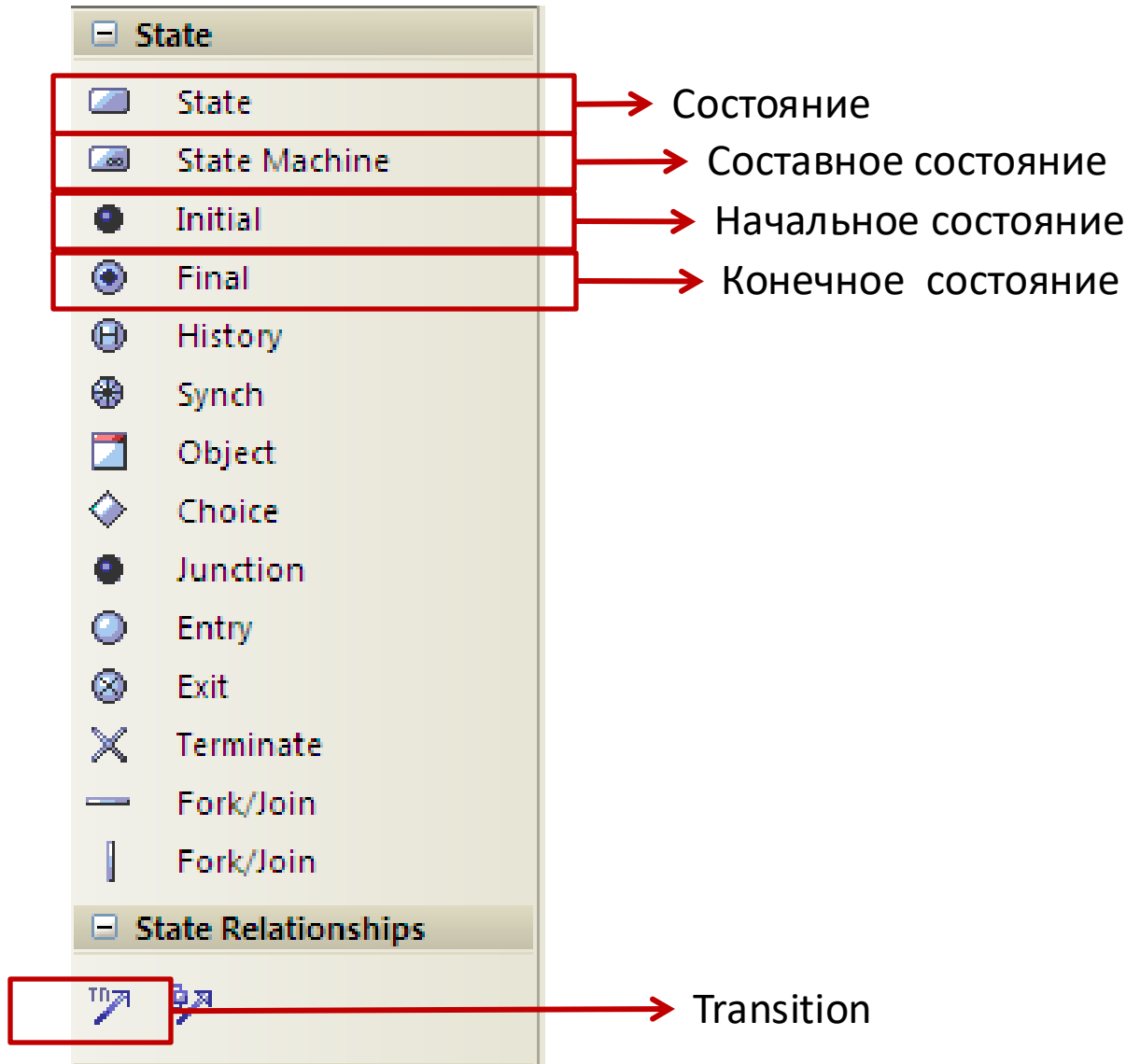
- Следует обязательно проверять, что никакие два перехода из одного состояния не могут сработать одновременно (требование отсутствия конфликтов у переходов). Наличие такого конфликта может служить признаком ошибки.
- В этом случае необходимо ввести дополнительные сторожевые условия либо изменить существующие, чтобы исключить конфликт переходов.

Создание диаграммы состояний

- New Diagram-> UML Behavioral-> State Machine



Создание диаграммы состояний



- Для создания диаграммы используется инструмент «State».

Спецификация состояния

State : State1

General Requirements Constraints Links Scenarios Files Tagged Values

Name:

Stereotype: ☐ Abstract

Author: Status:

Scope: Complexity:

Alias: Language:

Keywords:

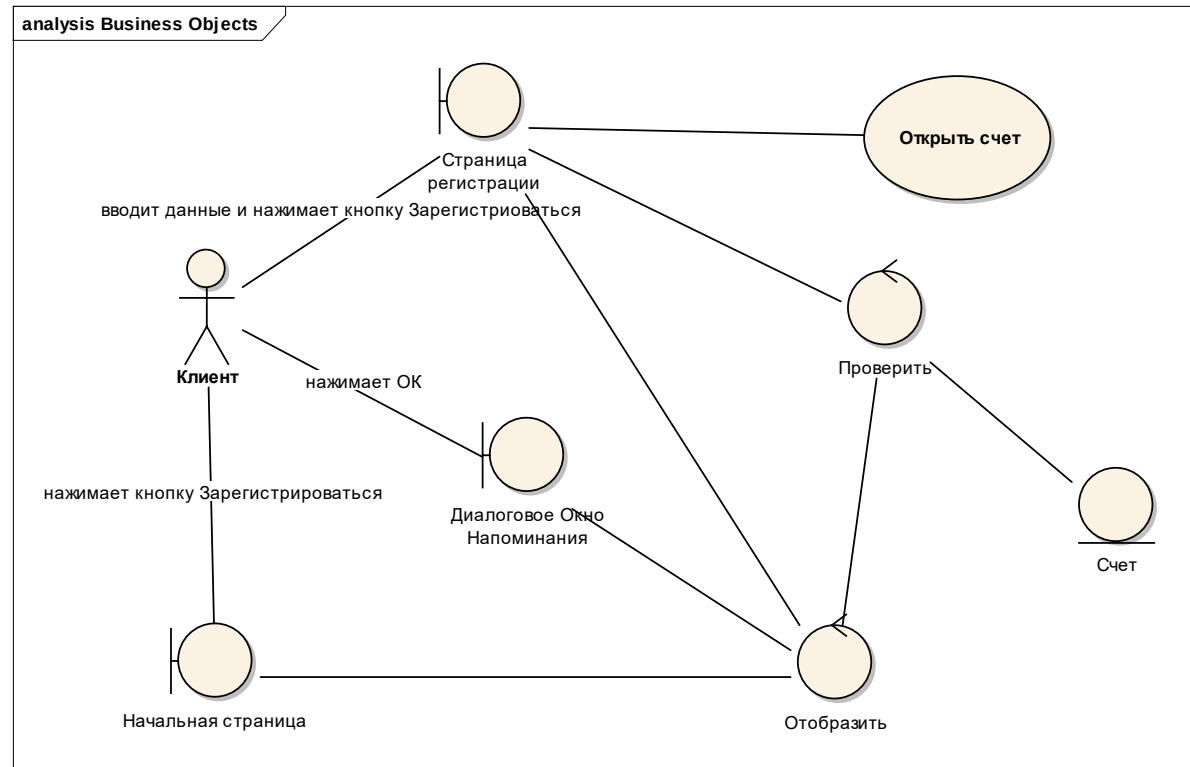
Phase: Version:

Notes:

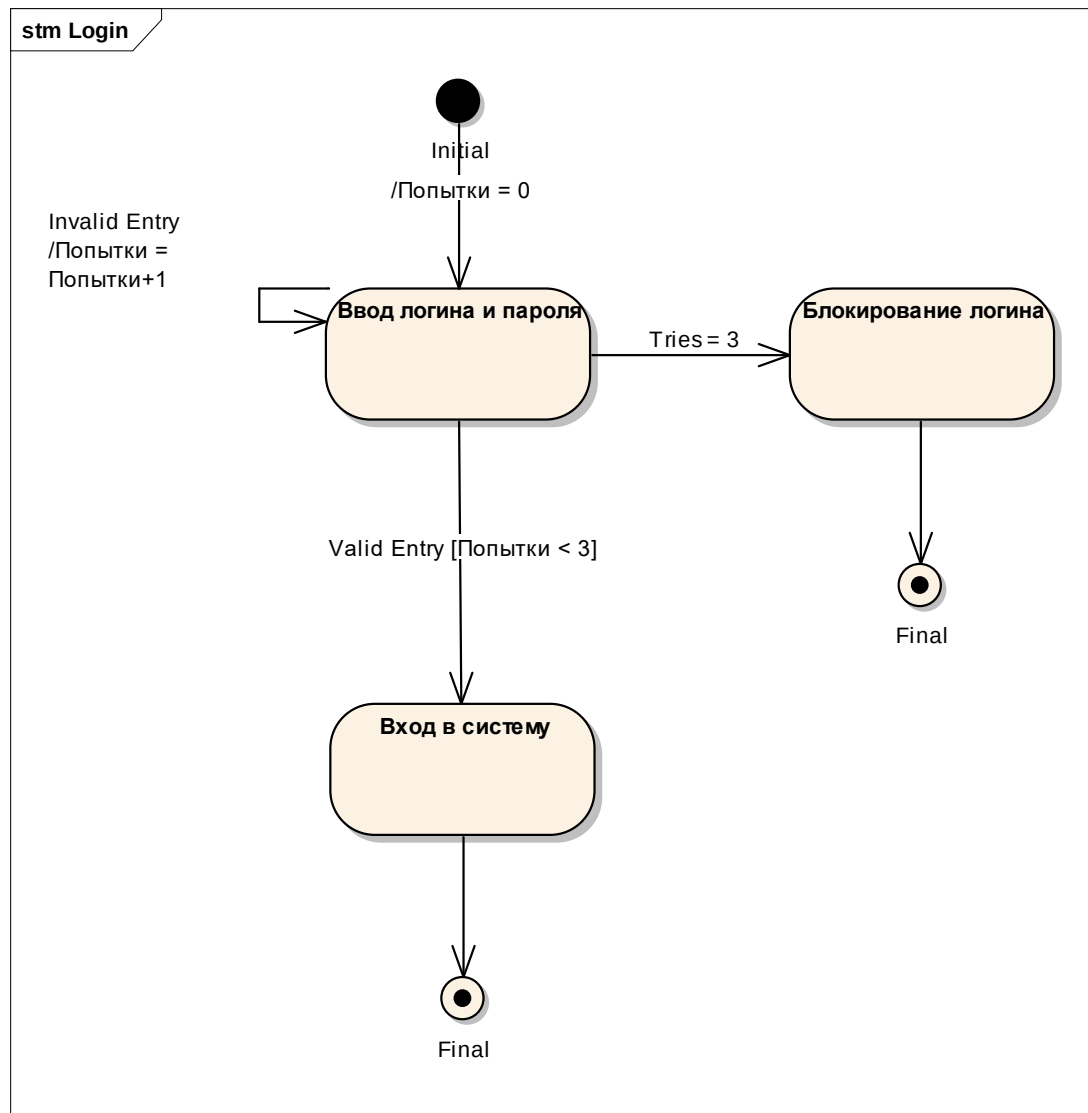
B I U A

Пример. Диаграмма пригодности «регистрация»

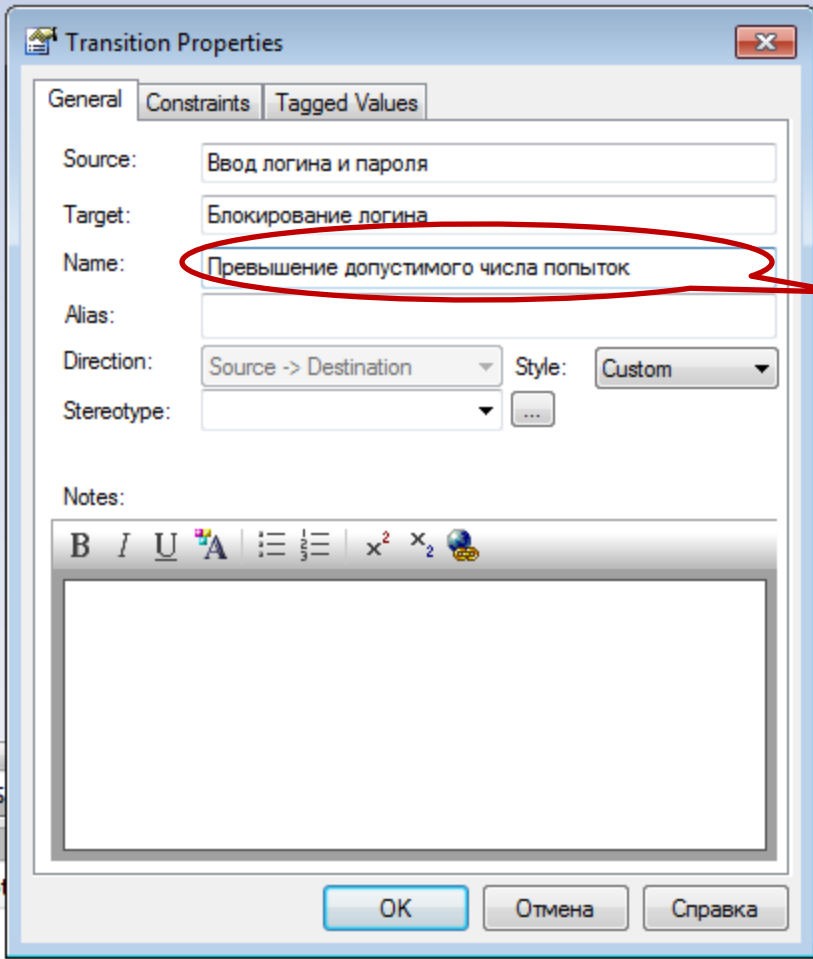
- **Альтернативные последовательности**
- Если **Клиент** нажимает кнопку **Новый счет** на **Странице регистрации**, то система вызывает прецедент **Открыть счет**.
- Если **Клиент** нажимает кнопку **Вспомнить** на **Странице регистрации**, то система выводит секретный вопрос, хранящийся для этого **Клиента**, в отдельном диалоговом окне. Когда **Клиент** щелкает в нем на кнопке **ОК**, открывается **Страница регистрации**.



Пример диаграммы состояний. Класс «Логин»



Спецификация связи Transition



The image shows a 'Transition Properties' dialog box with three tabs: 'General', 'Constraints', and 'Tagged Values'. The 'General' tab is active. It contains the following fields:

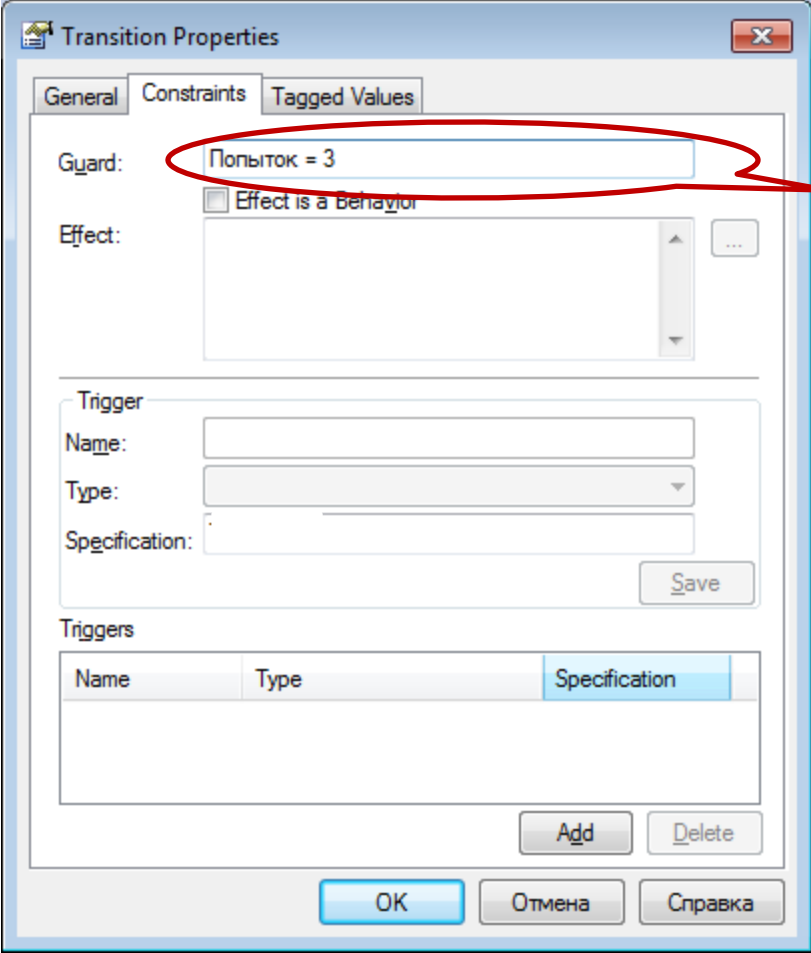
- Source: Ввод логина и пароля
- Target: Блокирование логина
- Name: Превышение допустимого числа попыток (circled in red)
- Alias:
- Direction: Source -> Destination
- Style: Custom
- Stereotype:

Below these fields is a 'Notes' section with a rich text editor toolbar (bold, italic, underline, text color, background color, bulleted list, numbered list, link, unlink, undo, redo) and a large empty text area.

At the bottom are three buttons: 'OK', 'Отмена' (Cancel), and 'Справка' (Help).

Имя события

Спецификация связи Transition

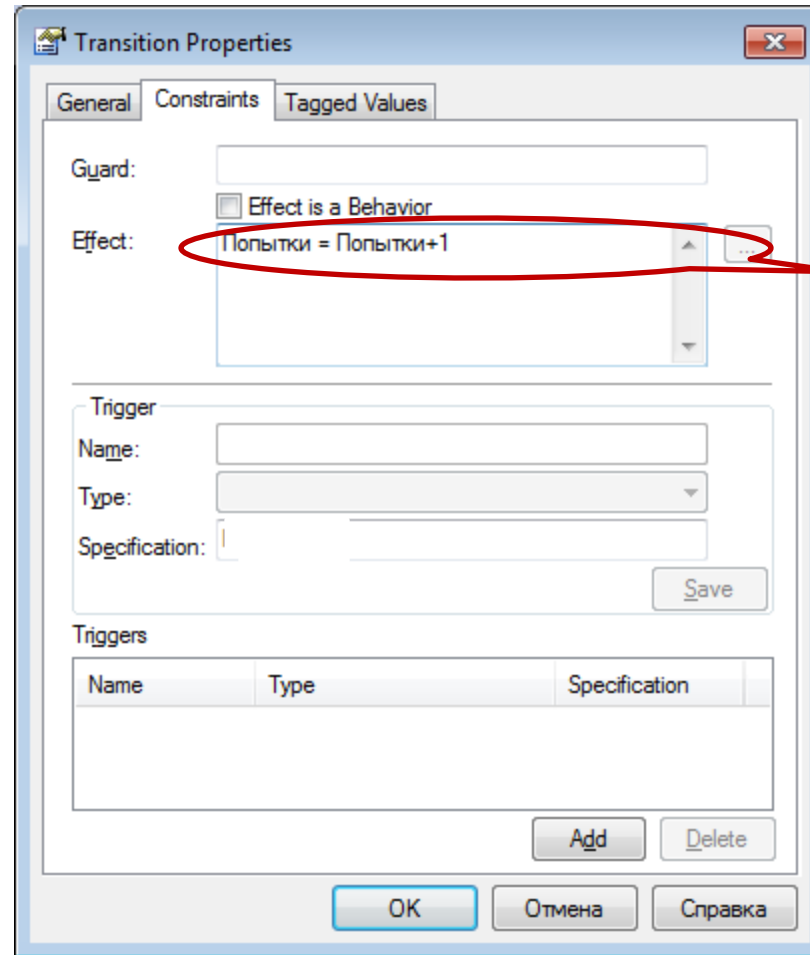


The image shows a 'Transition Properties' dialog box with three tabs: 'General', 'Constraints', and 'Tagged Values'. The 'General' tab is active. It contains a 'Guard' field with the text 'Попыток = 3', which is circled in red. Below the 'Guard' field is a checkbox labeled 'Effect is a Behavior'. The 'Effect' field is empty. Below these is a 'Trigger' section with fields for 'Name', 'Type', and 'Specification'. At the bottom of the dialog is a 'Triggers' table with columns 'Name', 'Type', and 'Specification'. The 'OK' button is highlighted in blue.

Name	Type	Specification
------	------	---------------

Сторожевое условие

Спецификация связи Transition



The image shows a 'Transition Properties' dialog box with three tabs: 'General', 'Constraints', and 'Tagged Values'. The 'General' tab is active. It contains a 'Guard' field, an 'Effect is a Behavior' checkbox, and an 'Effect' field. The 'Effect' field contains the text 'Попытки = Попытки+1', which is circled in red. Below the 'Effect' field is a 'Trigger' section with 'Name', 'Type', and 'Specification' fields. At the bottom is a 'Triggers' table with columns 'Name', 'Type', and 'Specification'. The table is currently empty. At the bottom of the dialog are 'OK', 'Отмена', and 'Справка' buttons.

Guard:

☐ Effect is a Behavior

Effect: Попытки = Попытки+1

Trigger

Name:

Type:

Specification:

Save

Triggers

Name	Type	Specification
------	------	---------------

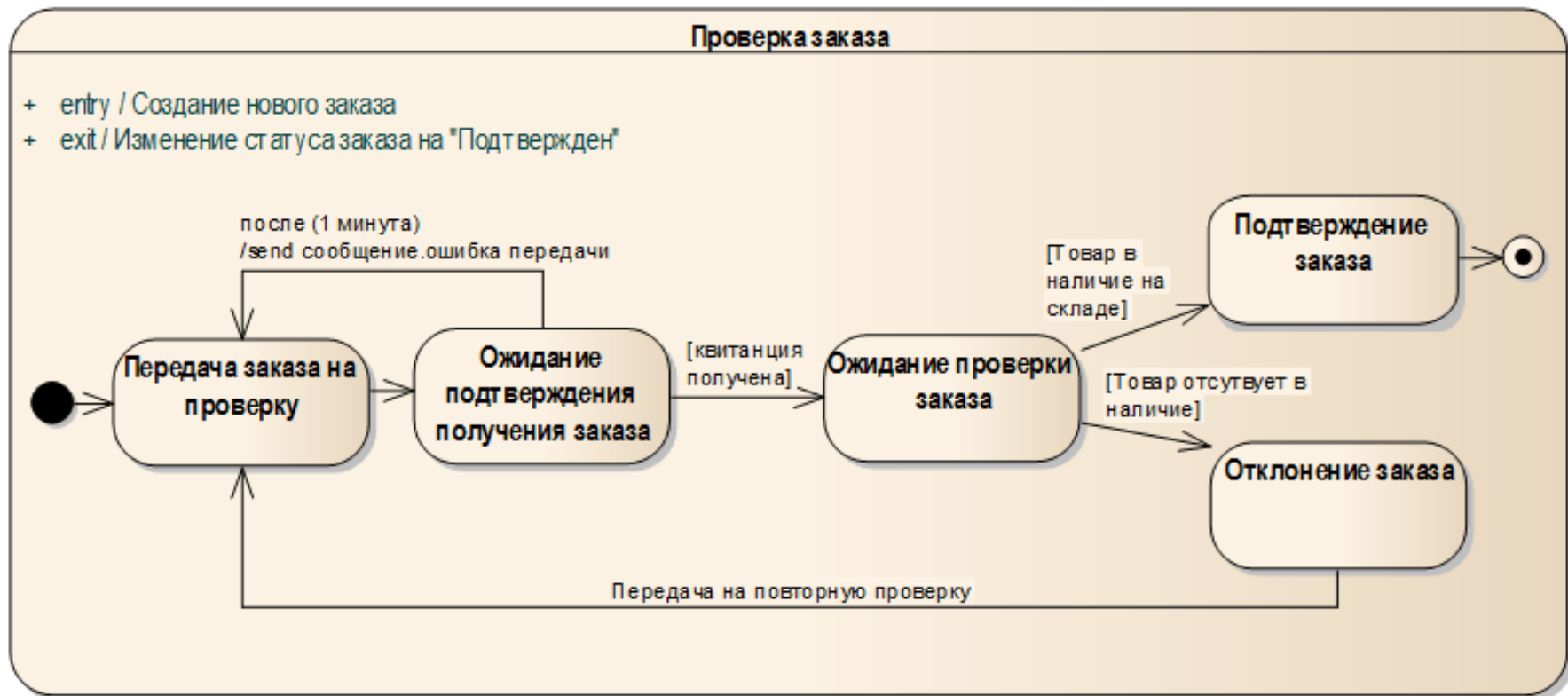
Add Delete

OK Отмена Справка

Выражение действия

Пример диаграммы состояний.

Класс «Проверка заказа»



P.S.

- В рамках курса рассмотрены не все элементы диаграммы состояний.
- Для детального ознакомления необходимо обратиться к первоисточнику.